



XRAY INDIVIDUAL PROJECT

VASCO HARTEVELD

Student name: Vasco Harteveld
Student number: 569923
Version: v0.2

Table of Contents

1	Introduction.....	3
2	Requirements.....	4
3	Analysis.....	5
3.1	Patient Administration.....	5
3.1.1	Context diagram.....	5
3.1.2	Use case diagram and descriptions.....	5
3.2	Central Acquisition.....	5
3.2.1	Context diagram.....	5
3.2.2	Use case diagram and descriptions.....	5
3.3	Geometry.....	5
3.3.1	Context diagram.....	5
3.3.2	Use case diagram and descriptions.....	5
3.4	Xray Generator.....	5
3.4.1	Context diagram.....	5
3.4.2	Use case diagram and descriptions.....	5
4	Static Software Design.....	6
4.1	Patient Administration.....	6
4.2	Central Acquisition.....	6
4.3	Geometry.....	6
4.4	Xray Generator.....	6
5	Dynamic Software Design.....	7
5.1	Patient Administration.....	7
5.2	Central Acquisition.....	7
5.3	Geometry.....	7
5.4	Xray Generator.....	7
6	Hardware Design.....	8
7	Verification.....	9
7.1	Patient Administration.....	9
7.2	System.....	9
7.3	Geometry.....	9
7.4	Xray Generator.....	9
7.5	Communication.....	9
8	Validation.....	10

1 Introduction

Give here a short introduction to this document. What is the intention of this document? Who are the intended readers of this document? Don't repeat here the description of the Xray project, just refer to the assignment.

In this project I will be creating a mock X-ray scanner in C/C++. This document is intended for future expansion, and potential forks of this project. I will be making a custom administration system, communicating with a hardware system of communicating modules. The modules will consist of a main communication Arduino, serving as a middleman between the other systems and the Linux storage system. There will be the Linux system with user input/output communicating with the Arduino's. And finally, there will be an Arduino with a mock dose system using an LED to represent an xray firing and an LDR to measure dosage. The Arduino's will use a custom I²C communication protocol to communicate to each other. The protocol will be focused on functions used solely in this project. We have 12 learning weeks for this project, using iterative product development split into sprints of 2-3 weeks. That means every sprint will have a different goal, to expand and change the previous implementation. Because of limited initial knowledge, the product will change drastically throughout its development process.

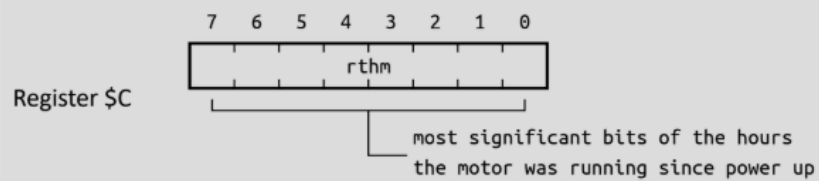
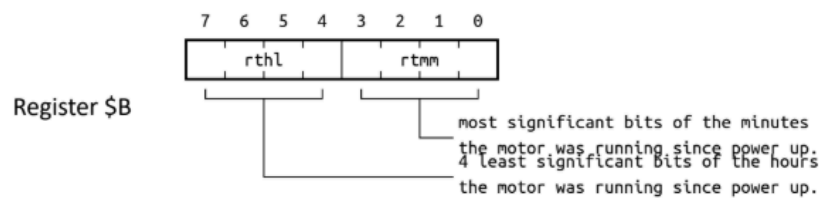
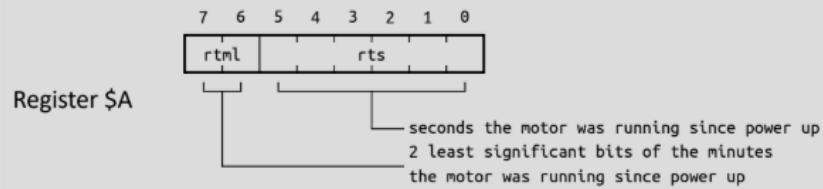
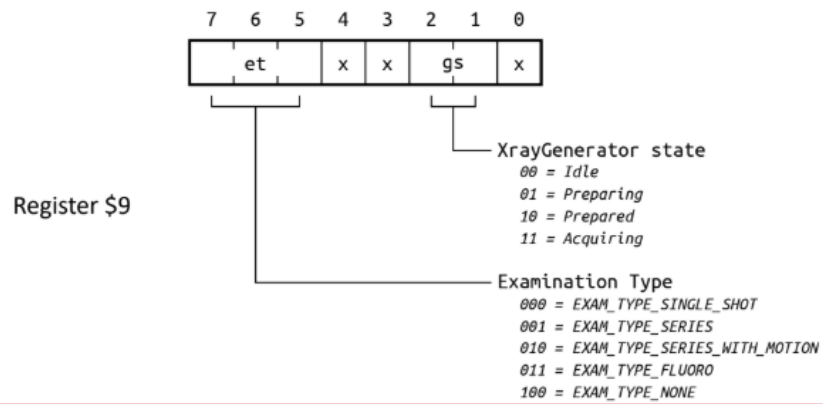
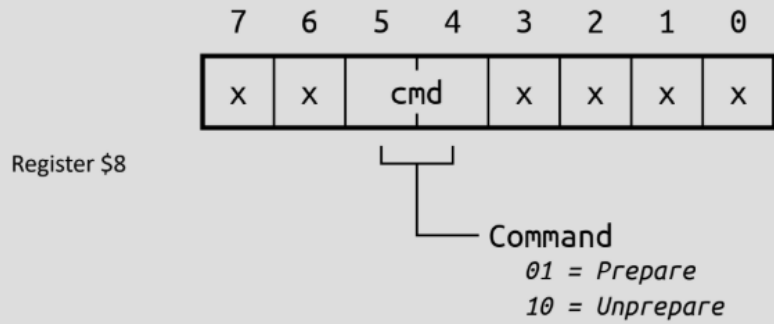
2 Requirements

Category	Requirement Number	Description
PATADMIN	1	At startup the following actions are performed (in the given order): • A patient dose administration is created. If an administration is stored in persistent memory (i.e. file) then the administration is filled with that stored data. Otherwise, an administration is created that only contains 1 patient, named JohnDoe (JohnDoe is always present in the administration). • PatientDoseAdministration tries, only once, to connect with CentralAcquisition. On success it transits from the not connected state to the connected state. • Selected examination type is set to EXAM_TYPE_NONE • The selected patient is set to JohnDoe Rationale for JohnDoe: in case of an emergency patient of which the credentials are unknown, still a patient needs to be selected. In such a case JohnDoe is selected as patient and must therefore always be present.
PATADMIN	2	A patient can always be set as the selected patient, unselecting a patient is not possible.
PATADMIN	3	At shutdown the following actions are performed (in the given order) before ending the application: a. If a patient named JohnDoe is stored in the administration then that patient is removed. b. If the patient admin still contains one or more patients, then all patient data is stored in persistent memory (i.e. file). c. If PatientDoseAdministration is connected to CentralAcquisition then that connection is broken down.
PATADMIN	4	It is always possible to add a patient in the administration. However, the name of the patient must be unique, is at most 80 characters long and does not equal JohnDoe.
PATADMIN	5	It is not possible to delete a patient when the patient to be deleted equals the selected patient or if the patient is JohnDoe, otherwise it is always possible.
PATADMIN	6	Selection of an examination type is only possible when the system is connected to CentralAcquisition and CentralAcquisition was not acquiring images. The following examination types can be selected: EXAM_TYPE_SINGLE_SHOT EXAM_TYPE_SERIES EXAM_TYPE_SERIES_WITH_MOTION EXAM_TYPE_FLUORO EXAM_TYPE_NONE
PATADMIN	7	When dose data is received it will be added to the selected patient. After that, the selected examination type is set to EXAM_TYPE_NONE (and passed to CentralAcquisition).
PATADMIN	8	PatientAdministration runs on a linux platform, and the administration needs to be implemented as an Abstract Data Type (ADT).

PATADMIN	9	The UI of the PatientAdministration application is a menu-based console application.
CentralACQ	0	Central Acquisition is the main commanding Arduino, and the master in the I ² C communication
CentralAcq	1	At startup CentralAcquisition is in the NotConnected state and waits only for the connect message, examination type is EXAM_TYPE_NONE.
CentralAcq	2	When Connect message is received it enters the Idle substate of the Connected state
CentralAcq	3	Only in the Idle substate the reception of a select examination type is handled, and an acknowledgement is replied. When not in the Idle state a not acknowledged is replied. On success, the received examination type is passed to Geometry and XrayGenerator
CentralAcq	4	When in the Idle substate the Prepare button changes to pressed and the examination type is not equal to EXAM_TYPE_NONE, it requests Geometry and XrayGenerator to prepare for acquisition and enters the preparing substate.
CentralAcq	5	When in the preparing substate it checks if both Geometry and XrayGenerator are in the prepared state, when that is the case, it enters the prepared substate and switches on the Prepared Lamp.
CentralAcq	6	When in the preparing substate for more than 1 second, it sends an unprepared message to Geometry and XrayGenerator and enters the Idle substate
CentralAcq	7	When in the preparing substate and the Prepare button changes to not pressed, it sends an unprepared message to Geometry and XrayGenerator and enters the Idle substate
CentralAcq	8	When in the prepared substate the X-ray button changes to pressed the Acquiring substate is entered and SAN_XRAY_ENABLED is set high and remains high as long as the X-ray button is pressed. The Prepared Lamp is switched off and the Acquiring Lamp on
CentralAcq	9	When in the acquiring substate and the X-ray button changes to not pressed it performs the following actions: • switch off the Acquiring Lamp • SAN_XRAY_ENABLED is set low • set the examination type to EXAM_TYPE_NONE and passes that type to Geometry and XrayGenerator • retrieves the received dose from XrayGenerator and passes it to PatientAdministration • enter the Idle substate
CentralAcq	10	When in any of the Connected substates the Disconnect message is received it performs the following actions: • switch off all lamps • SAN_XRAY_ENABLED is set low • set the examination type to EXAM_TYPE_NONE • enter the NotConnected state

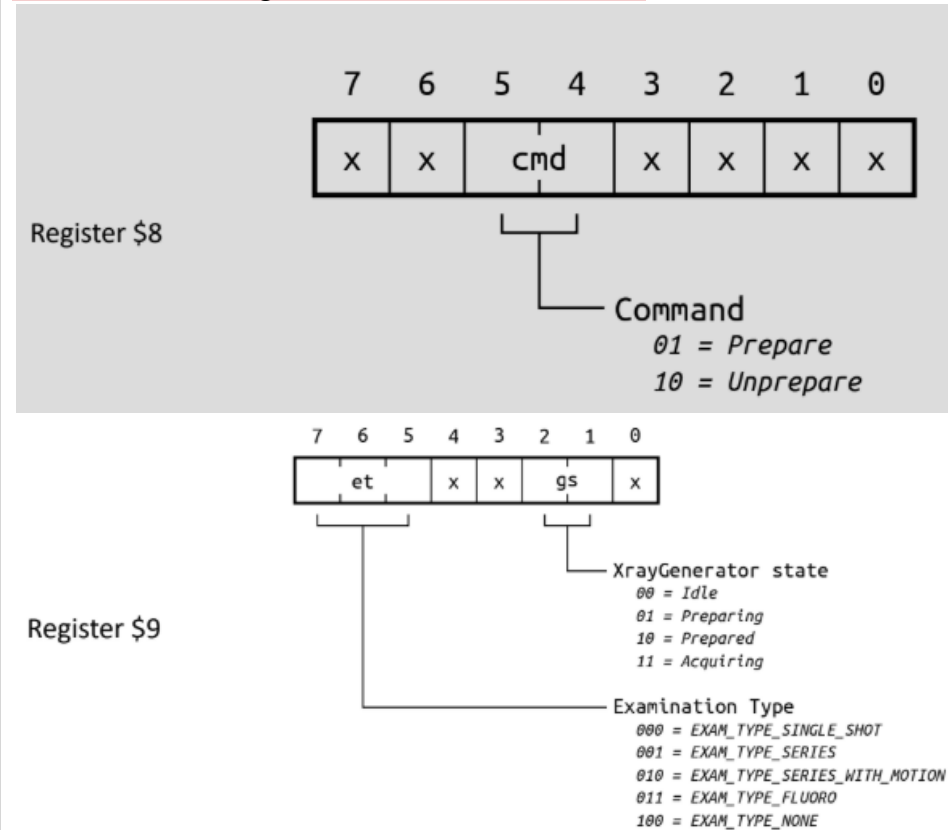
GEO	0	Geometry is the Arduino in charge of the motors and joystick, commanded by the Central Acquisition.
GEO	1	At startup Geometry is in the Idle state.
GEO	2	Motion is possible in any Geometry state.
GEO	3	The motor will rotate at the half of the maximum speed if the FixedSpeed button is pressed.
GEO	4	The motor will rotate at a speed depending on the position of the joystick speed if the joystick is not in the zero position.
GEO	5	When the FixedSpeed button is activated before the joystick, the joystick is ignored if the FixedSpeed is activated. The same for the other way around.
GEO	6	When prepare is received, it waits some time before going to the prepared state. The wait time mimics the prepare actions that need to be done. The wait time is a random value between [0.1, 0.4] seconds. But when the selected examination type is SERIES_WITH_MOTION it is between [0.3, 0.8] seconds. On reception of the prepare, geometry enters the Preparing state. After the wait time Geometry enters the Prepared state.
GEO	7	When in the preparing or prepared state and unprepare is received, it waits some time before going to the Idle state. The wait time mimics the unprepare actions that need to be done. The wait time is a random value between [0.0, 0.2] seconds.
GEO	8	When examination type equals EXAM_TYPE_SERIES_WITH_MOTION and the state is Prepared and SAN_XRAY_ENABLED is High and EnableMove changes from Low to High then and only then the state becomes acquiring and SAN_GEO_MOVING is set High, and remains High only as long as EnableMove is High and SAN_XRAY_ENABLED is High. EnableMove is high if and only if the FixedSpeed button or joystick is activated. When SAN_GEO_MOVING is changed from High to Low, transits from Acquiring to the Idle state.
GEO	9	Geometry has registers that can be read and written by CentralAcquisition. The registers contain information about the selected examination type, state of Geometry and the hours, minutes and seconds the motor was running since uptime. But also, a register in which a command (e.g. prepare) can be written.

Addresses of the registers and their definition:



XRAY	0	Xray Generator was the Arduino in charge of emitting Xrays (LED) and measuring them with an LDR.
XRAY	1	At startup XrayGenerator is in the Idle state.
XRAY	2	Generating X-ray is impossible when SAN_XRAY_ENABLED is Low or examination type is EXAM_TYPE_NONE.
XRAY	3	When in the idle state and prepare is received, it enters the preparing state. In the preparing state it sets the register that will contain the received dose to zero, it prepares all that is needed for the timing of the xray pulse(s). To mimic the unpredictable timing behavior of the prepare actions a random wait time between [0.1,w 0.2] seconds is added. After all preparing actions are done it enters the Prepared state.
XRAY	4	When in the preparing or prepared state and unprepare is received, it waits some time before going to the Idle state. The wait time mimics the unprepare actions that need to be done. The wait time is a random value between [0.2, 0.4] seconds.
XRAY	5	When in the preparing or prepared state and unprepare is received, it waits some time before going to the Idle state. The wait time mimics the unprepare actions that need to be done. The wait time is a random value between [0.2, 0.4] seconds.
XRAY	6	When in the prepared state and selected examination type is EXAM_TYPE_SERIES_WITH_MOTION and SAN_GEO_MOVING is High and SAN_XRAY_ACTIVE changes from Low to High the Acquiring state is entered.
XRAY	7	When in the Acquiring state and examination type is EXAM_TYPE_SINGLE_SHOT then one X-ray pulse is generated. The pulse has a duration of 10 milliseconds. The power of the pulse is at its maximum level.
XRAY	8	When in the Acquiring state and examination type is TYPE_SERIES or EXAM_TYPE_SERIES_WITH_MOTION then a series of X-ray pulses is generated. Each pulse has a duration of 10 milliseconds. The time between two pulses is fixed, 0.49 seconds. The power of the pulses is at its maximum level.
XRAY	9	When in the Acquiring state and examination type is EXAM_TYPE_FLUORO then a series of X-ray pulses is generated. Each pulse has a duration of 10 milliseconds. The time between two pulses is fixed, 0.24 seconds. The power of the pulses is at a 10th of the maximum level
XRAY	10	When in the Acquiring state and SAN_XRAY_ENABLED or SAN_GEO_MOVING changes from High to Low then the Idle state is entered
XRAY	11	When during a pulse SAN_XRAY_ENABLED or SAN_GEO_MOVING changes from High to Low then the pulse is completed before going to the Idle state.
XRAY	13	XrayGenerator has registers that can be read and written by CentralAcquisition. The registers contain information about the selected examination type, actual state of XrayGenerator, the number of pulses at max power generated since uptime and the

number of pulses at 10th max power since uptime. But also, a register in which a command (e.g. unprepare) can be written.
Addresses of the registers and their bit definition:



Register \$A...\$D Contains the number (uint32_t) of pulses at the maximum power since power up. \$A contains the most significant byte of that number, \$D the least significant byte.

Register \$E ...\$11 contains the number (uint32_t) of pulses at the 10th of the maximum power since power up. \$E contains the most significant byte of that number, \$11 the least significant byte.

At startup register \$9 has value 0x80, and the registers \$A ... \$11 and \$8 value 0x00.

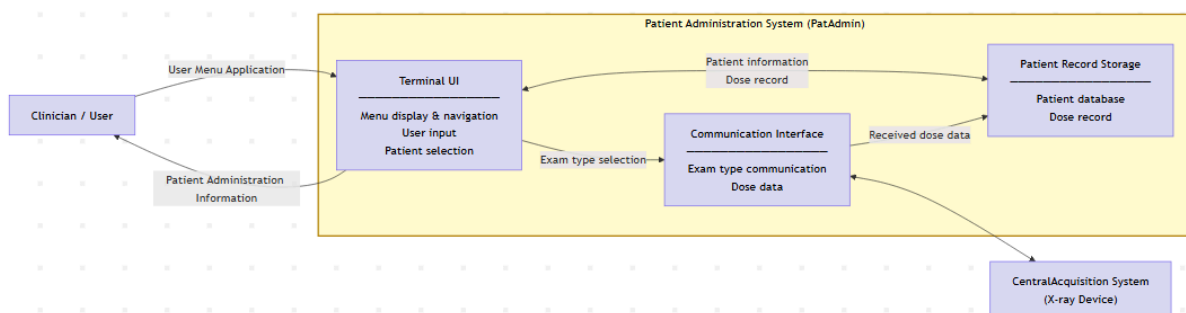
3 Analysis

In this chapter, you describe your analysis, which is done for the 4 components separately. For each component, you give a context diagram, a use case diagram, and for each use case a description. A use case description must be given in the following format:

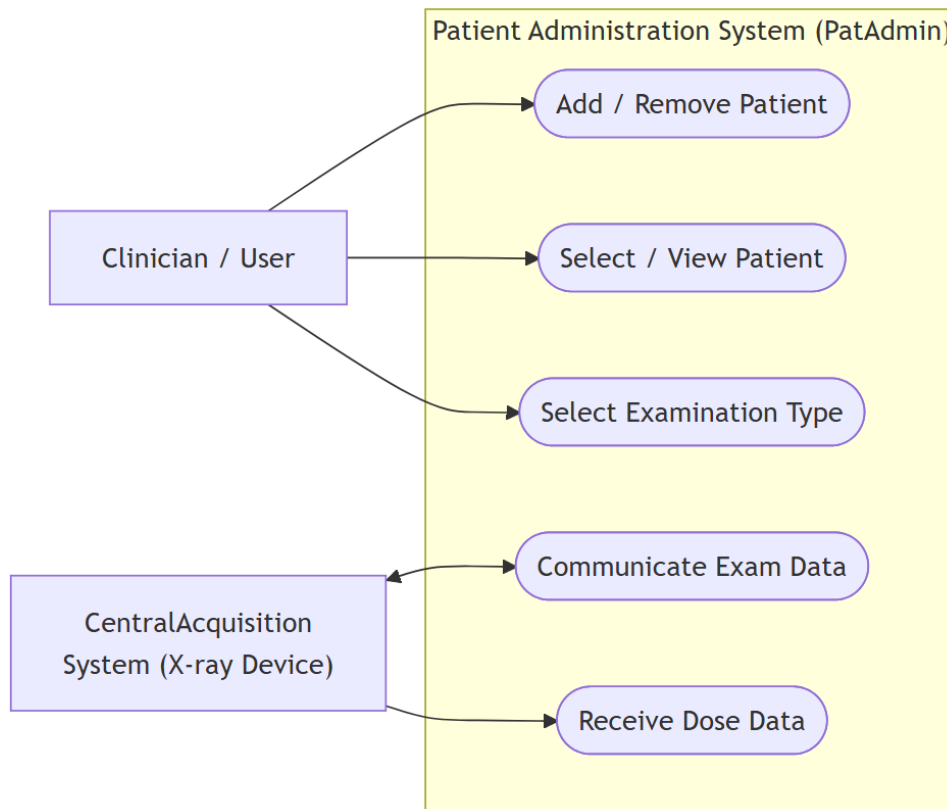
<i>Name</i>	<i>Add here the name of the use case as given in the diagram.</i>
<i>Description</i>	<i>Give here a short description of the use case.</i>
<i>Actor(s)</i>	<i>Add here the actors that are involved in the use case, using the names as given in the diagram.</i>
<i>Trigger</i>	<i>Describe here what triggers (activates) the use case.</i>
<i>Standard</i>	<i>Describe here the execution steps.</i>
<i>Flow</i>	<i>Describe here the alternative execution behaviour (leave empty when not applicable).</i>
<i>Alternative</i>	<i>Describe here the alternative execution behaviour (leave empty when not applicable).</i>
<i>Flow</i>	<i>When you have multiple alternative flows, then add table rows.</i>

3.1 Patient Administration

3.1.1 Context diagram



3.1.2 Use case diagram and descriptions



Name	Add Patient
Description	For adding new patients into the database system
Actor(s)	Clinician/User
Trigger	An input into the terminal application
Standard Flow	Safety checks (is the patient already present? Can they be added), then allocate memory for the added patient. Calculate an index from the hash function, then add the patient into the head of the hash table, and link it to the following elements.

Name	Remove Patient
Description	For removing existing patients from the database system
Actor(s)	Clinician/User
Trigger	An input into the terminal application
Standard Flow	Safety checks (is the patient present? Can they be removed), then link the rest of the linked list depending if they are in the head, tail or middle of a chain, and free memory.

Name	Select Patient
Description	For viewing patient dose records and adding a dose
Actor(s)	Clinician/User
Trigger	An input into the terminal application

Standard Flow	Safety checks (is the patient present?), find the address of the patient structure, and save them into a separate variable.
Alternative Flow	If a patient is selected, the clinician can then view the patient's data.
Alternative Flow	If dose data is incoming, it is saved to the selected patient.

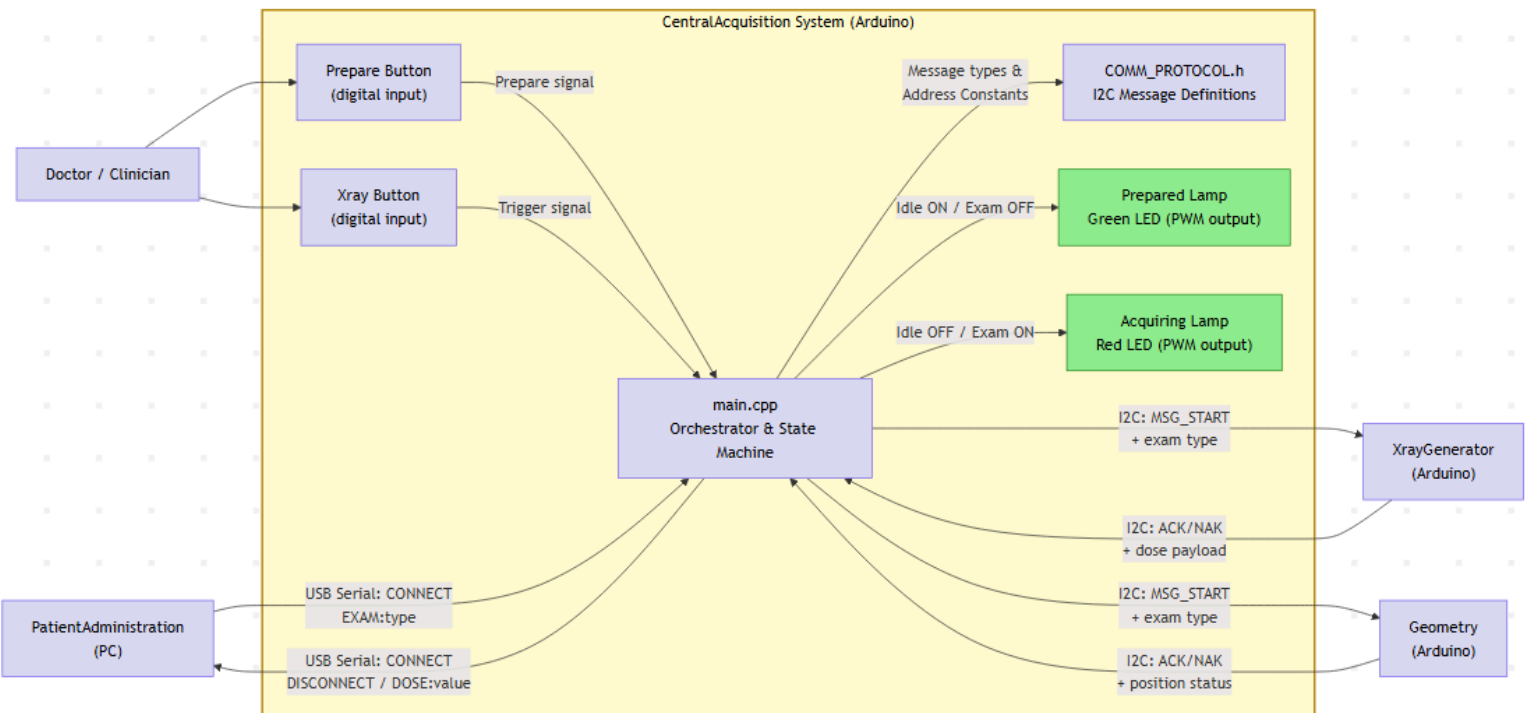
Name	Select Examination Type
Description	For inputting/altering the current examination type
Actor(s)	Clinician/User
Trigger	An input into the terminal application
Standard Flow	Safety checks (Is the application connected to the Arduino system?), send a message to the serial port with the correct start/end symbol and syntax.
Alternative Flow	If Arduino system is disconnected, inform the user

Name	Communicate Exam Data
Description	For communicating the desired exam type
Actor(s)	Central Acquisition System, Patient Administration System
Trigger	Select Examination Type being run from the terminal application.
Standard Flow	Check if the message is exam data, check if a patient is selected, then add the dose into the patient's data.

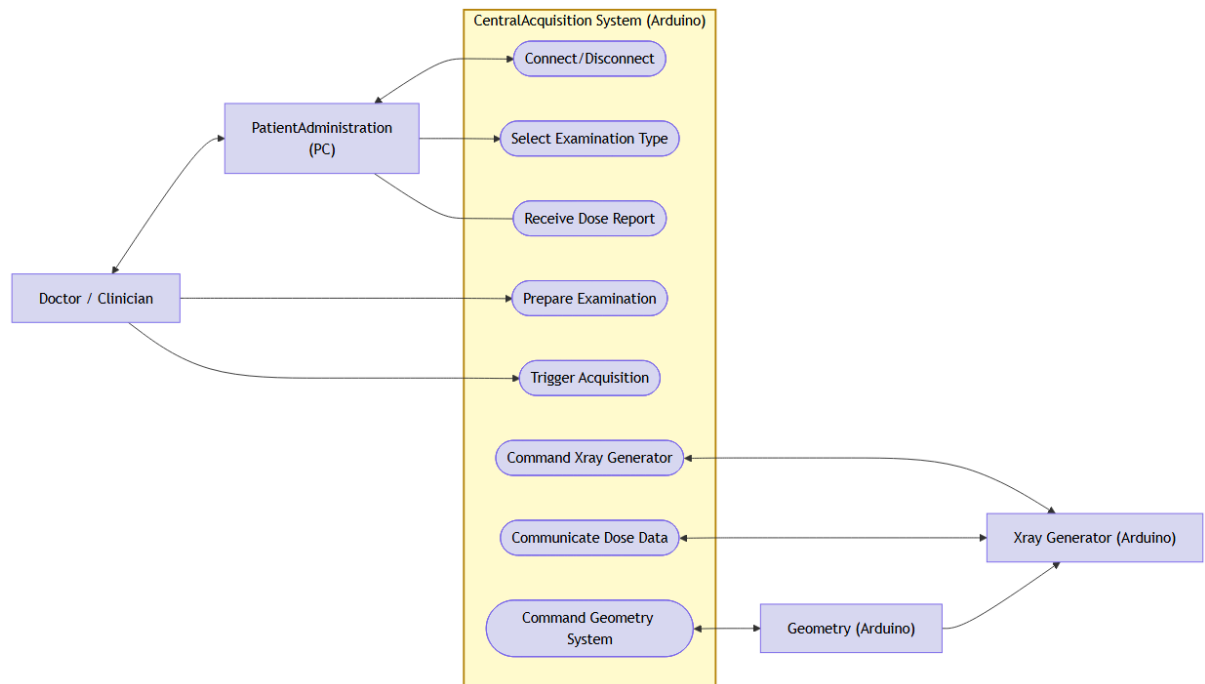
Name	Receive Dose Data
Description	For communicating the last measured dosage after an exam has occurred
Actor(s)	Central Acquisition System, Xray Generator
Trigger	A specific message type being received on the Serial Bus
Standard Flow	Check if the message is exam data, check if a patient is selected, then add the dose into the patient's data.

3.2 Central Acquisition

3.2.1 Context diagram



3.2.2 Use case diagram and descriptions



Name	Connect/Disconnect
Description	For initiating and breaking down the communication from the Patient Administration to the Central Acquisition system.
Actor(s)	Patient Administration, Central Acquisition System
Trigger	Starting or stopping the terminal application.
Standard Flow	Sending a specific message on the serial port when no exam is prepared.

Name	Select Examination Type
Description	For communicating the current examination type
Actor(s)	Clinician/User
Trigger	An input into the terminal application
Standard Flow	Safety checks (Is the application connected to the Arduino system?), send a message to the serial port with the correct start/end symbol and syntax.
Alternative Flow	If Arduino system is disconnected, inform the user

Name	Receive Exam Command
Description	Communicating the new chosen exam type
Actor(s)	Patient Administration, Central Acquisition System, Doctor
Trigger	A specific message type being received on the Serial Bus, followed with a mapped exam identifier
Standard Flow	A Doctor selects an exam type in the terminal application, the Patient Administration checks if the Arduino system is present,

	then sends a specific message on the serial port, with a number representing a specific exam type.
--	--

Name	Transmit Dose Report
Description	For communicating the last measured dosage after an exam has occurred
Actor(s)	Central Acquisition System, Xray Generator
Trigger	A specific message type being received on the Serial Bus
Standard Flow	Check if the message is exam data, check if a patient is selected, then add the dose into the patient's data.

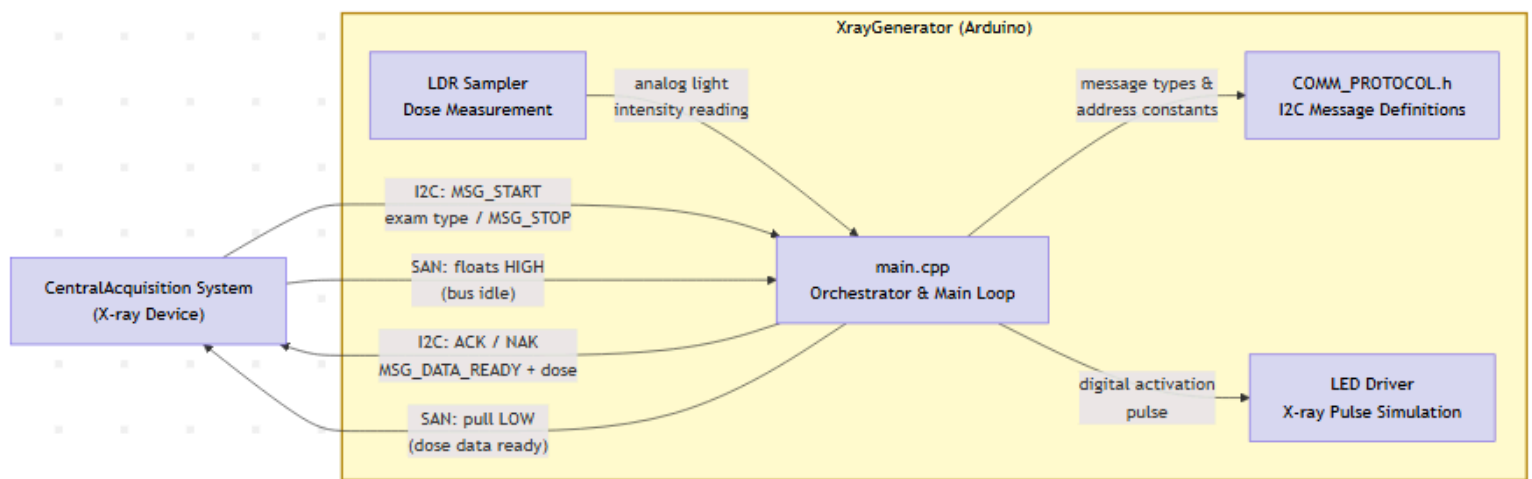
3.3 Geometry

3.3.1 Context diagram

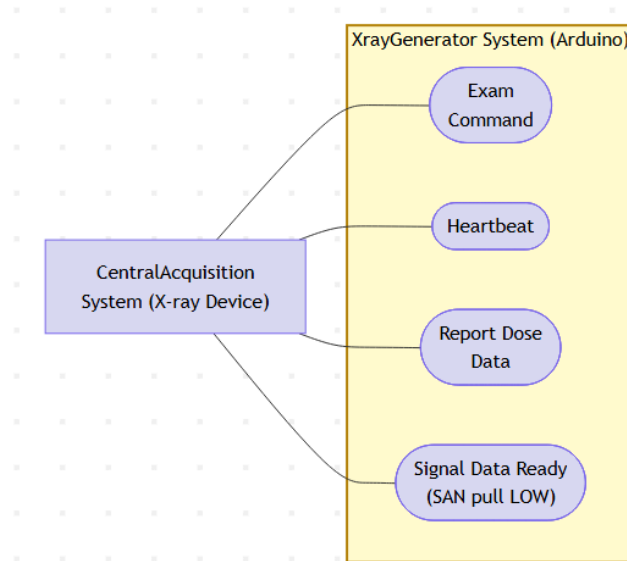
3.3.2 Use case diagram and descriptions

3.4 Xray Generator

3.4.1 Context diagram



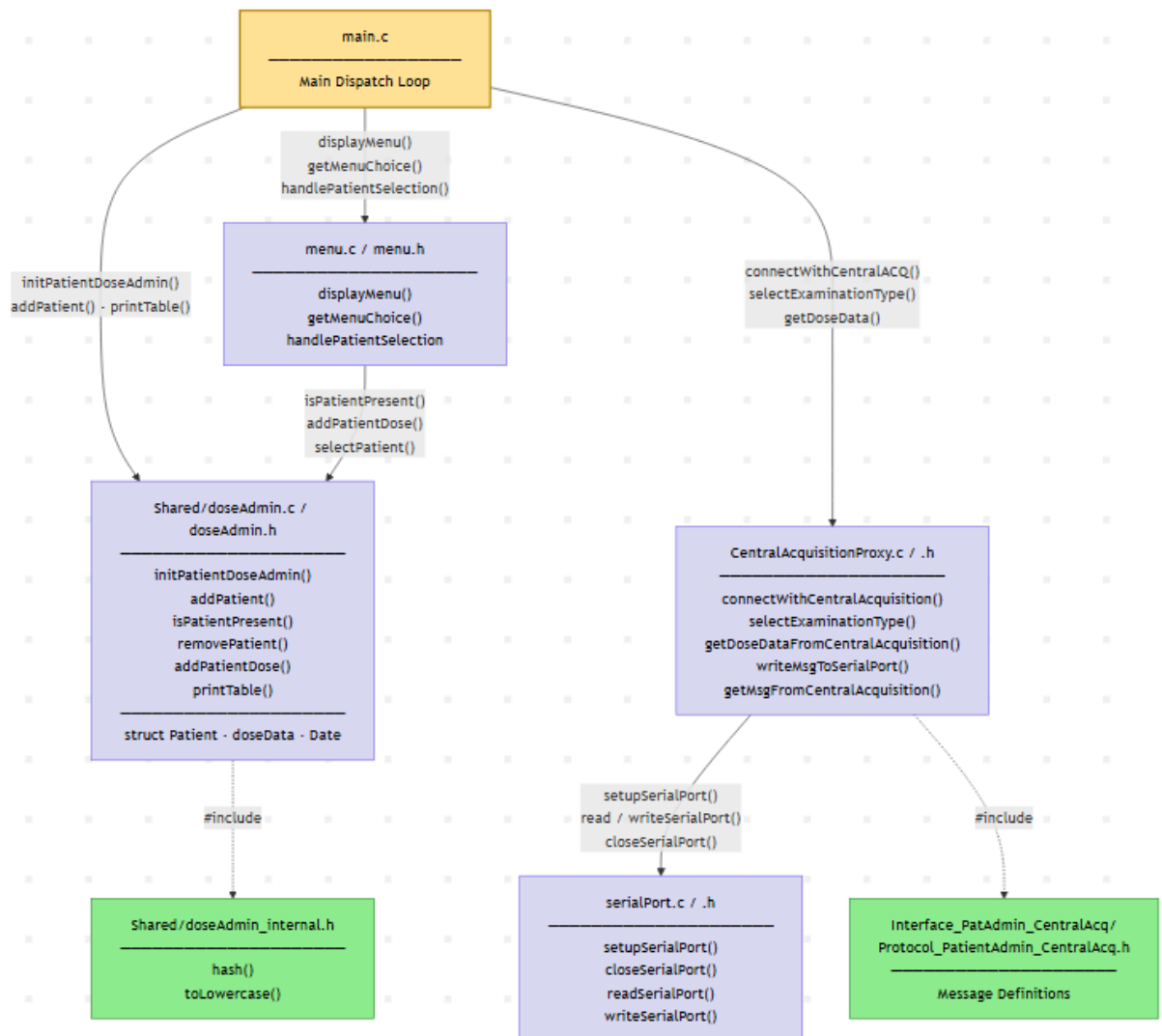
3.4.2 Use case diagram and descriptions



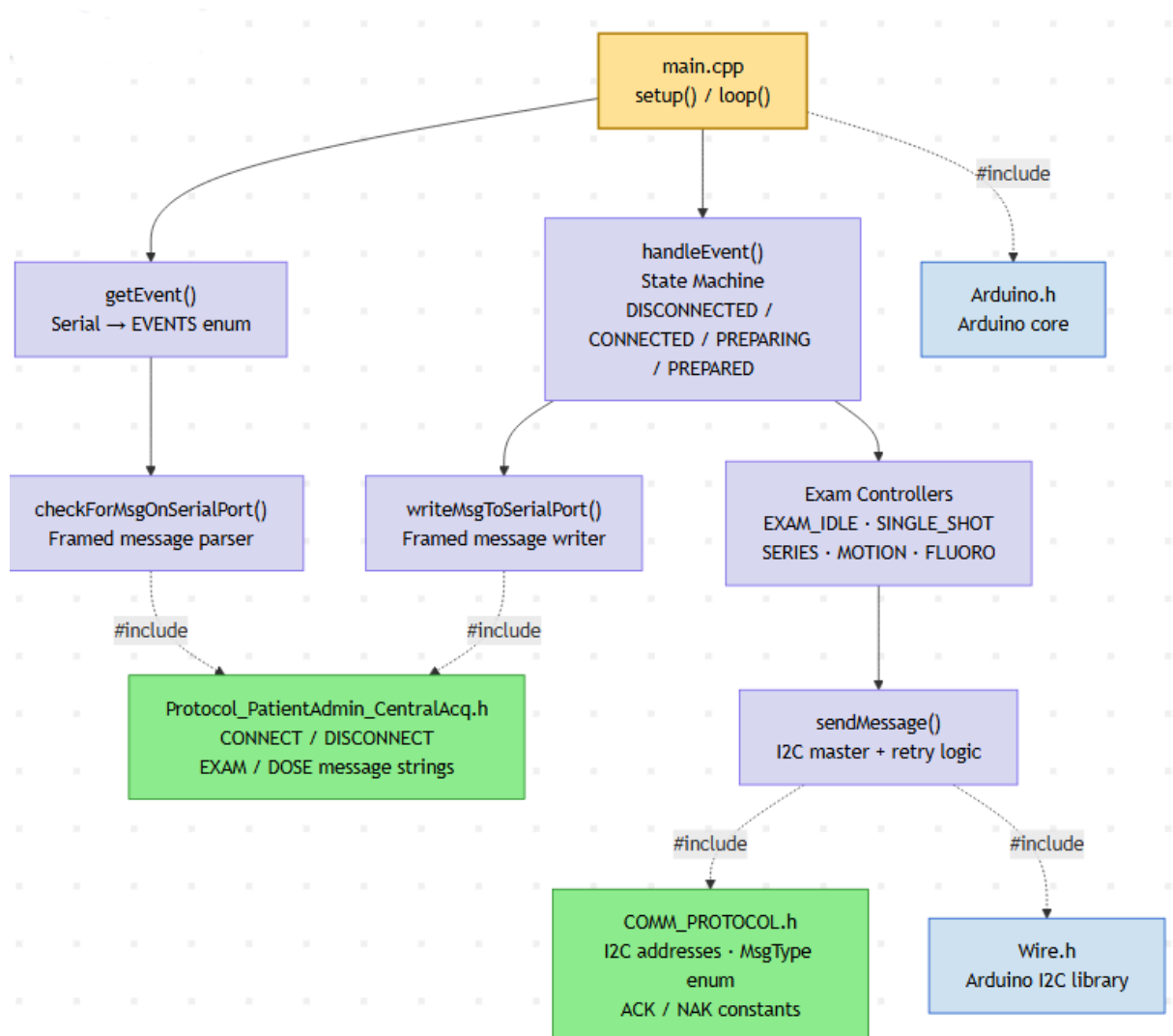
4 Static Software Design

In this chapter, you describe the static structure of your software; this is done for the 4 components separately. For each component, you give a module structure diagram. That diagram must match the actual status of the sprint delivery, so a previously given diagram may need an update.

4.1 Patient Administration

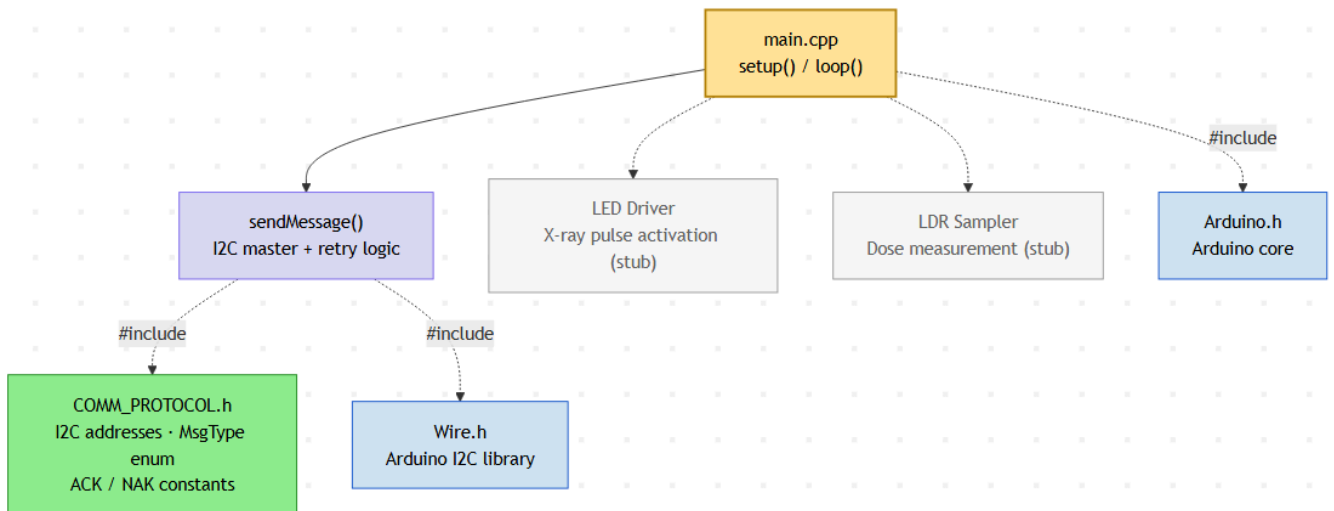


4.2 Central Acquisition



4.3 Geometry

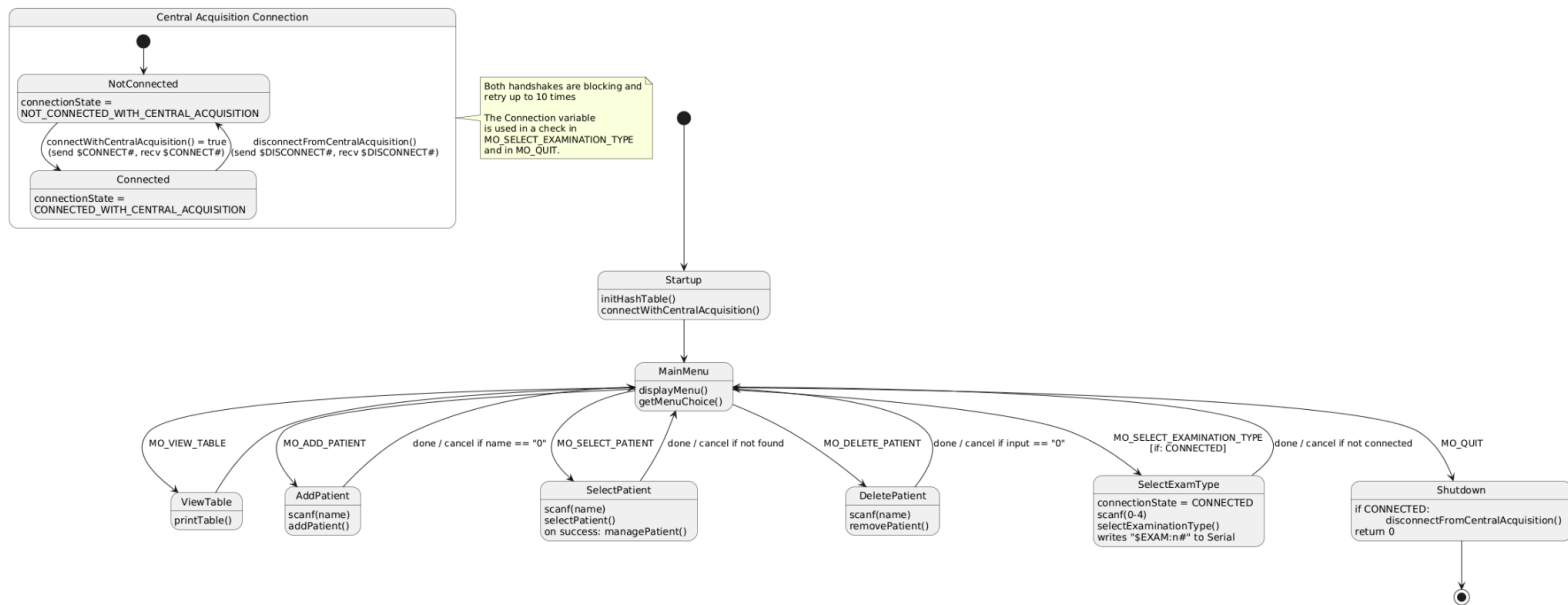
4.4 Xray Generator



5 Dynamic Software Design

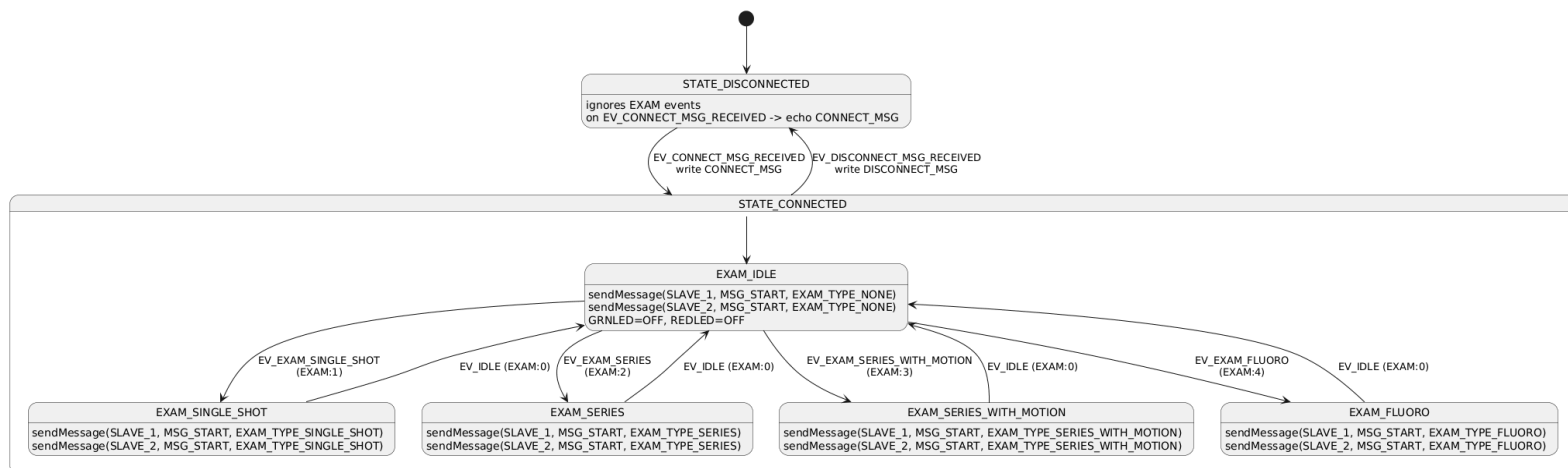
5.1 Patient Administration

Patient Administration - State Machine

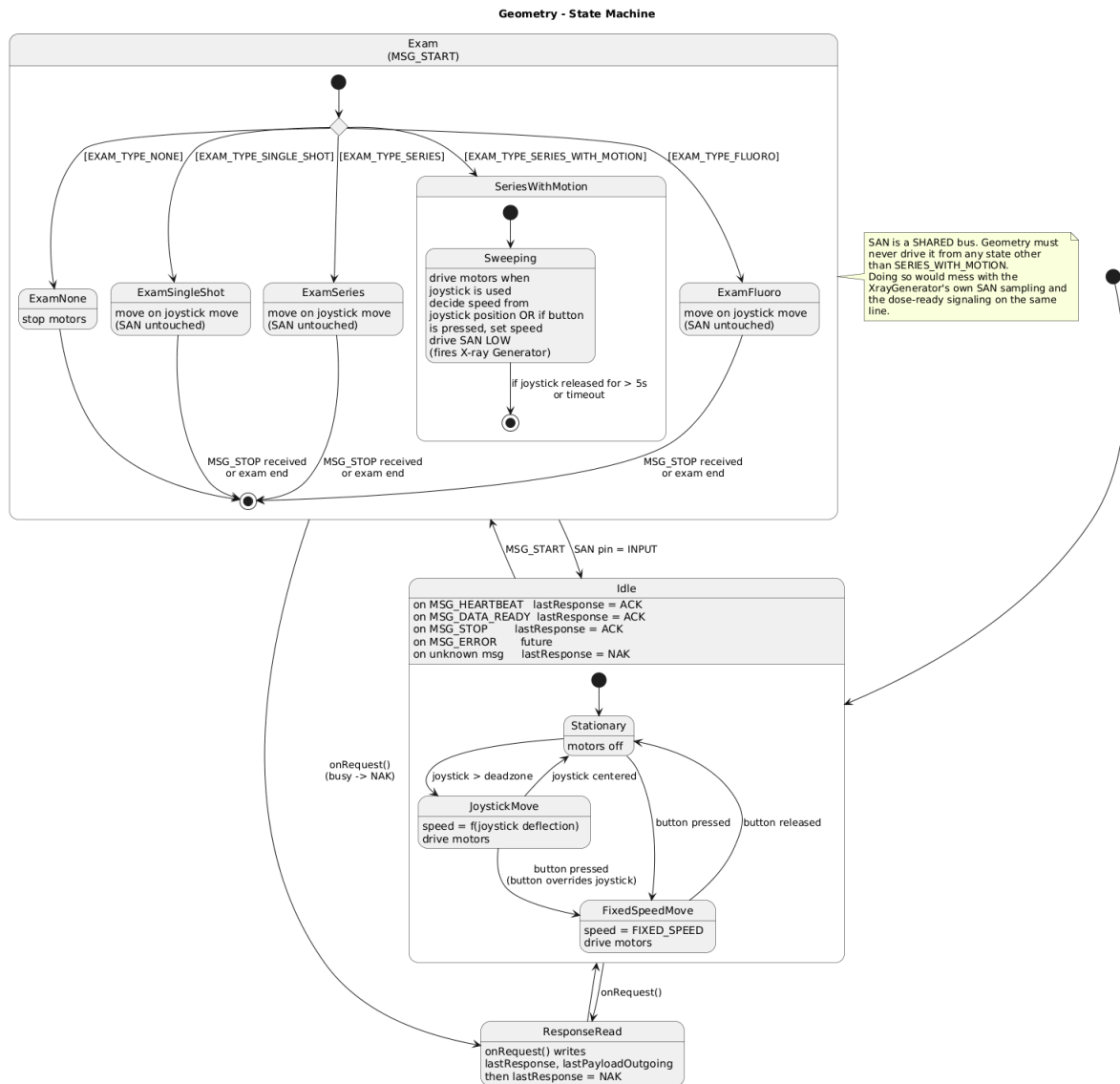


5.2 Central Acquisition

Central Acquisition - State Machine

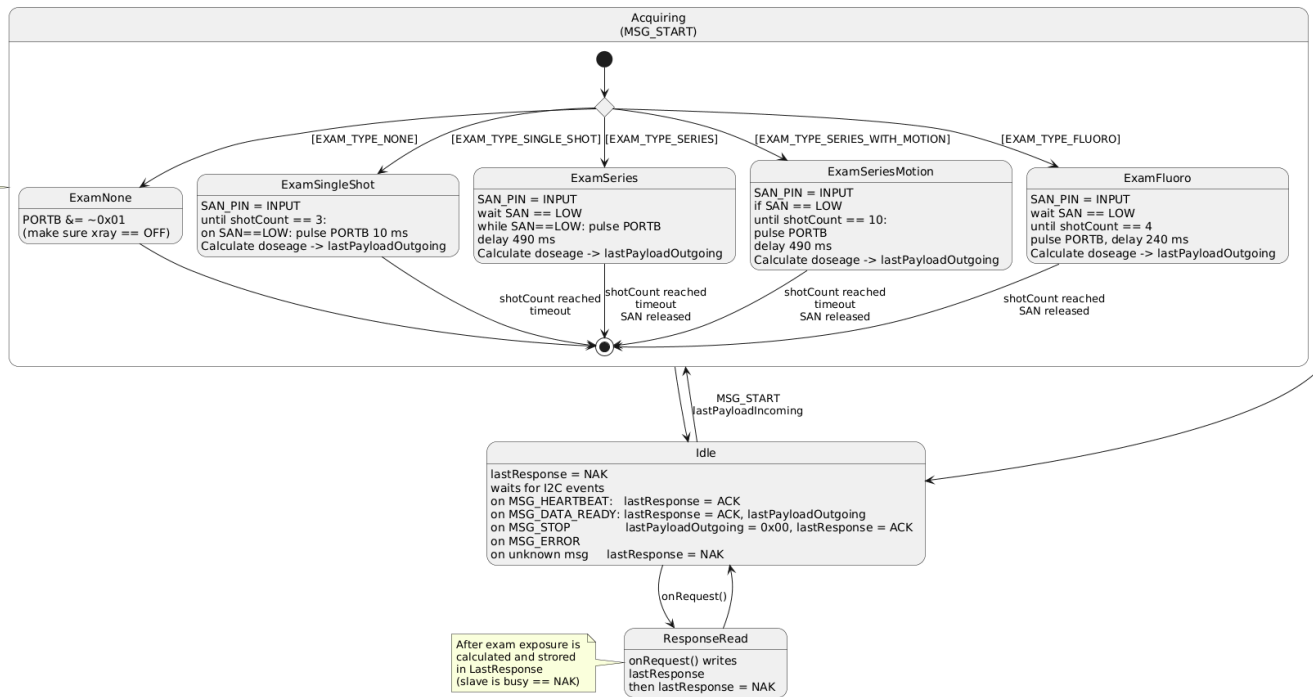


5.3 Geometry

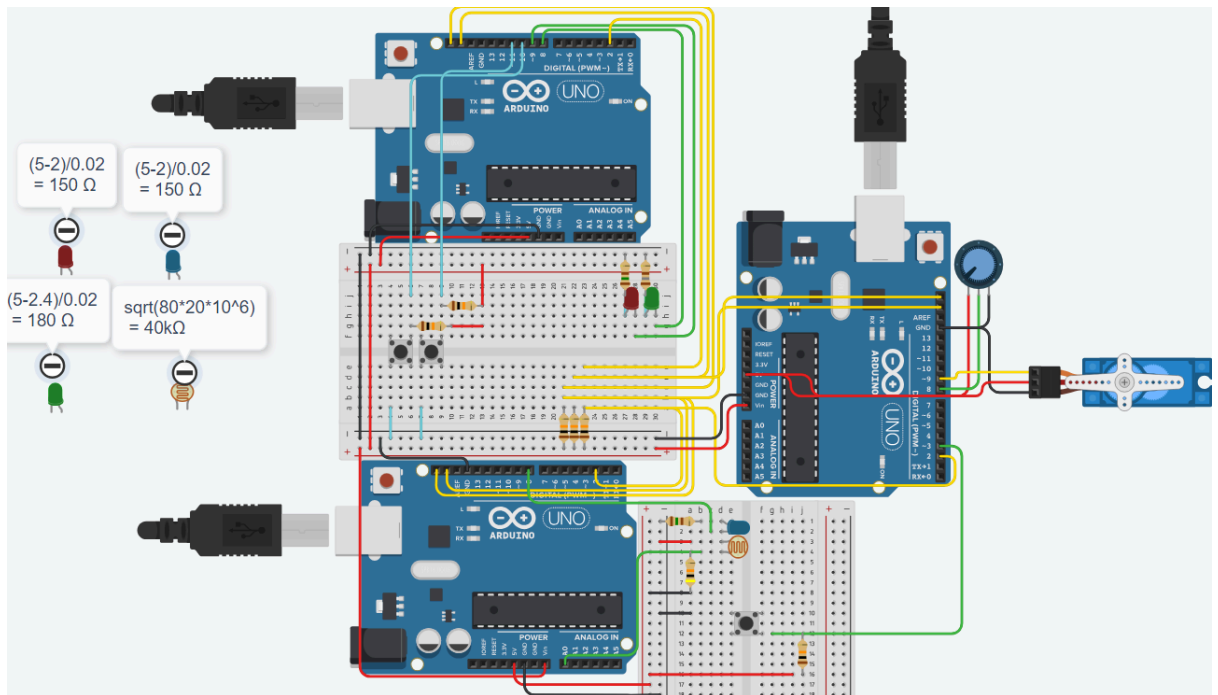


5.4 Xray Generator

X-ray Generator - State Machine



6 Hardware Design



7 Protocol Design

In this chapter, you describe your communication protocol between the PatientAdministration (PC) and CentralAcquisition.

8 Verification

8.1 Patient Administration

In this section, you list the names of your test cases. From the test case name, it must be clear what is tested and what the expected result should be. Add a screenshot showing the actual result.

All functions given in the header of doseAdmin need to be tested.

8.2 System

In this section, you describe testcases that cover the complete product. These testcases are executed manually, and at least the PatientAdministration and CentralAcquisition components are involved. However, testcases in which all components are involved need to be present as well. In total, I expect at least 7 testcases.

You can describe a system testcase in, for instance, the following table format:

<i>Test name: Preparing happy flow</i>			
<i>Test description: this test verifies the preparing behavior of CentralAcquisition and</i>			
<i>Precondition: the complete system is wired, patient admin is empty</i>			
<i>tep</i>	<i>Manual action</i>	<i>Expected result</i>	<i>ass/ Fail</i>
	<i>Add patient (name according spec)</i>	<i>Patient is added {Patadmin 4}</i>	
	<i>Select the new patient</i>	<i>Patient is selected {Patadmin 2}</i>	
	<i>....</i>		

When a test step failed, describe below the table what went wrong.

Also note the used requirement annotation in the expected result column.

8.3 Communication

In some sprints, you are asked to show what is happening on communication lines.

Add here screenshots for the I2C and socket communication (logic analyzer, Wireshark).

9 Validation

In this chapter, you add one or more videos of the system you made. Also, describe the results of the demo you gave to your clients (i.e., teachers). This chapter is filled in for the final sprint only.